

NeuGIER: Neural Graphics and Inverse Rendering (exercise sheet 1)

In this exercise, we familiarize ourselves with the differentiation of integrals with continuous and discontinuous integrand. You will implement both forward-mode and reverse-mode automatic differentiation in a programming language of your own choice. Further, you apply the Leibniz integral rule on paper to differentiate under the integral sign. The **deadline for this assignment is May 7**, where you will be asked to show us your solution in the lab exercise.

Assignment 1 [7 Points] (Implement Forward-Mode Automatic Differentiation)

The necessary theory for this assignment is discussed in the lecture on April 14.

Implement forward-mode automatic differentiation using dual numbers in any programming language you want. You are not allowed to use libraries that already contain automatic differentiation or dual numbers. The goal of the exercise is to implement those yourself.

- Create a data type that represents a dual number $(a, b) = a + b\epsilon$. Your data type should store a primal a and a tangent b . (1 Pt)
- Implement the following common arithmetic operators for your dual number type. You may either use operator overloading (e.g., $+$ or $-$) or you can simply write functions (e.g., `add`).
 - addition ($+$ operator) (0.5 Pts)
 - subtraction ($-$ operator) (0.5 Pts)
 - multiplication ($\times$ operator) (0.5 Pts)
 - division ($\div$ operator) (0.5 Pts)
 - sine function (`sin` function) (0.5 Pts)
 - exponential function (`exp` function) (0.5 Pts)
- To confirm the correctness of your implementation, evaluate the partial derivatives of the function $f(x, y) = \left(\sin\left(\frac{x}{y}\right) + \frac{x}{y} - e^y\right) \cdot \left(\frac{x}{y} - e^y\right)$ using your code at the location $x = 1, y = 2$ and see if you get the following result:

$$f(1, 2) = 44.1563, \quad \frac{\partial f(x, y)}{\partial x} \Big|_{x=1, y=2} = -9.6722, \quad \frac{\partial f(x, y)}{\partial y} \Big|_{x=1, y=2} = 103.101$$

Note that this is the computation graph that was used in the lecture. The lecture slides list all intermediate results, which can be useful for debugging. (1 Pt)

- When you hand in your solution in the lab exercise, we will ask you to evaluate a different function $g(x, y)$ that is given to you by us on that day. (2 Pts)

Assignment 2 [8 Points] (Implement Reverse-Mode Automatic Differentiation)

The necessary theory for this assignment is discussed in the lecture on April 14.

Implement reverse-mode automatic differentiation in any programming language you want. You can choose whether you want to implement it recursively or iteratively, and you can choose whether you want to implement a gathering or scattering approach. As before, you are not allowed to use libraries that already contain automatic differentiation. The goal of the exercise is to implement it yourself.

- Create a data type that represents a node in the computation graph. Each node i stores: a primal v_i , an adjoint $\bar{v}_i$, and a list of children (or parents) $C_i = \{(j, \frac{\partial v_j}{\partial v_i})\}$ for the gathering (or scattering) of the adjoint. The list contains both the node j and the corresponding edge weight $\frac{\partial v_j}{\partial v_i}$. (1 Pt)
- Implement the following common arithmetic operators for your data type. You may either use operator overloading (e.g., $+$ or $-$) or you can simply write functions (e.g., `add`).
 - addition ($+$ operator) (0.5 Pts)
 - subtraction ($-$ operator) (0.5 Pts)
 - multiplication ($\times$ operator) (0.5 Pts)
 - division ($\div$ operator) (0.5 Pts)
 - sine function (`sin` function) (0.5 Pts)
 - exponential function (`exp` function) (0.5 Pts)

Note that each operation not only computes the primal a , it also adds edges in the computation graph. In a scattering implementation, the children receive a reference to their parents, while in a gathering implementation, the parents receive a reference to their children.

- Implement a backward propagation that traverses the computation graph in reverse order to compute the adjoints $\bar{a}$. You can implement this iteratively or recursively. For example, a gathering-based implementation computes the adjoint $\bar{v}_i$ of parent node i from the edges found in its list of children C_i (i.e., dependent variables):

$$\bar{v}_i = \sum_{(j, \frac{\partial v_j}{\partial v_i}) \in C_i} \bar{v}_j \frac{\partial v_j}{\partial v_i} \quad (1)$$

At the end of the backward propagation, the adjoints of your input variables store their corresponding partial derivatives. (1 Pt)

- To confirm the correctness of your implementation, evaluate the partial derivatives of the function $f(x, y) = \left(\sin\left(\frac{x}{y}\right) + \frac{x}{y} - e^y\right) \cdot \left(\frac{x}{y} - e^y\right)$ using your code at the location $x = 1, y = 2$ and see if you get the following result:

$$f(1, 2) = 44.1563, \quad \frac{\partial f(x, y)}{\partial x} \Big|_{x=1, y=2} = -9.6722, \quad \frac{\partial f(x, y)}{\partial y} \Big|_{x=1, y=2} = 103.101$$

Again, this is the computation graph that was used in the lecture. The lecture slides list all intermediate results, which can be useful for debugging. (1 Pt)

- When you hand in your solution in the lab exercise, we will ask you to evaluate a different function $g(x, y)$ that is given to you by us on that day. (2 Pts)

Assignment 3 [5 Points] (Differentiation under the Integral Sign)

The necessary theory for this assignment is discussed in the lecture on April 21.

Automatic differentiation has trouble with integrands that contain discontinuities or when the integration bounds depend on the parameter that is differentiated for. In such cases, the Leibniz integral rule is applied. Compute the following derivative on paper for $\phi > 0$:

$$\frac{d}{d\Phi} \int_{2\Phi}^{4\Phi} f(x, \Phi) dx \quad \text{with} \quad f(x, \Phi) = \begin{cases} \frac{x}{\Phi} - 1 & x < 3\Phi \\ 1 & x \geq 3\Phi \end{cases} \quad (2)$$

and thereby split the integral into the following terms:

$$\frac{d}{d\Phi} \int_{2\Phi}^{4\Phi} f(x, \Phi) dx = f_{\text{int}} + f_{\text{limit}} + f_{\text{edge}} \quad (3)$$

where f_{int} is the interior integral, f_{limit} accounts for changes in the integration limits, and f_{edge} accounts for discontinuities of the integrand.

Have fun :)